20. tcpdump Basics for Engineers

20. tcpdump Basics for Engineers

Phase: Phase 11 — Observability and troubleshooting
Estimated time: 2 days
Slug: `20-tcpdump-basics`

Learning objectives

- Capture with filters
- Write/read pcap files
- Read flags and ports
- Minimize capture blast radius

Prerequisites

Complete prior lessons. Prefer a Linux VM for `ip`/`ss`/`nft` examples; macOS works for many `curl`/`dig` drills. Type every command yourself and keep a lab notes file.

Why this matters

Metrics say error rate up; app logs are clean. A 30-second `tcpdump` with a tight BPF often shows RSTs, retransmits, or the wrong VIP before you guess.

Core concepts

What you must be able to explain:

- Capture with filters
- Write/read pcap files
- Read flags and ports
- Minimize capture blast radius

Mental model for `20-tcpdump-basics` (exercise kind `tcpdump_port`):

- Prefer **measurable** checks (exit codes, ICMP replies, DNS rcode, HTTP status, socket state) over vibes.
- Separate **reachability** (can packets flow?) from **correctness** (right service, cert, route, identity).
- Ask: *which layer's tool falsifies my current hypothesis?* Then run that tool.
- Record the exact command that proved the root cause — that becomes your playbook snippet.

Worked commands

Run these on a lab host. Linux VM preferred for `ip`/`ss`/`tcpdump`/`nft`; macOS is fine for many `curl`/`dig`/`ping` drills. Your addresses and timings will differ — match the *shape* of the output and the conclusion you draw.

Example 1

```
sudo tcpdump -ni any -c 5 udp port 53
```

Sample output:

```
12:01:01.1 IP 10.0.0.5.53122 > 1.1.1.1.53: UDP, length 32
12:01:01.2 IP 1.1.1.1.53 > 10.0.0.5.53122: UDP, length 48
```

Interpretation: BPF `port 53` captures DNS. Direction and lengths hint query vs response. Always capture only systems you may test.

Example 2

```
sudo tcpdump -ni any -c 8 'tcp port 443 and (tcp[tcpflags] & tcp-syn != 0)'
```

Sample output:

```
... Flags [S], ...
... Flags [S.], ...
```

Interpretation: SYN and SYN-ACK prove handshake progress. SYN with no reply suggests drop/filter/blackhole along the path.

Example 3

```
tshark -r /tmp/lab.pcap -Y 'dns' -c 5 2>/dev/null || echo 'save a pcap first:
sudo tcpdump -w /tmp/lab.pcap -c 20 port 53'
```

Sample output:

```
1 0.000000 10.0.0.5 → 1.1.1.1 DNS 74 Standard query A example.com
```

Interpretation: Display filters (`-Y`) slice a saved capture without re-running traffic — safer on busy hosts.

Mini-lab

1. Re-type every worked command for **tcpdump Basics for Engineers** and paste outputs into lab notes.
2. Introduce one controlled failure in the lab (wrong DNS, firewall drop, bad route) and recover using only this lesson's tools.
3. Write a 5-line playbook: symptom → first command → expected healthy output.
4. Complete the exercise stubs; then explain each function's return value without looking.

Failure modes

Symptom	Likely cause	Next check
Capturing on prod without policy	See common mistakes	Re-run lesson commands
Huge unfiltered captures filling disks	See common mistakes	Re-run lesson commands
Sniffing secrets on shared networks	See common mistakes	Re-run lesson commands
Timeout with no RST	Silent drop / blackhole	tcpdump + compare path

Study tip for tcpdump Basics for Engineers

Before each new command, write a one-line hypothesis. If the result falsifies it, update the hypothesis instead of stacking random tools. Note equivalents across OS (`ip` ↔ `ifconfig`/`route`, `ss` ↔ `netstat`). Re-run two commands from this lesson tomorrow from memory and explain the sample outputs out loud.

Evidence habit

When pages are down, prefer evidence (`ip route get`, resolver answer, packet capture, listener list) over intuition. Document **command → output → conclusion** in a lab notebook so the same path is reusable on-call. Change one variable at a time; write a one-line hypothesis before each new tool.

Common mistakes

- Capturing on prod without policy
- Huge unfiltered captures filling disks
- Sniffing secrets on shared networks

Exercises

Work in `exercises/20-tcpdump-basics/`. Read that folder's `README.md` before coding.

```
cd exercises/20-tcpdump-basics
chmod +x test.sh exercise.sh 2>/dev/null || true
./test.sh
```

What you will build

Edit `exercise.sh` and implement `tcpdump_dns_filter`, `tcpdump_numeric_flag`, `tcpdump_write_flag`. Each function is a tiny retrieval drill for **tcpdump Basics for Engineers** — return the exact tokens/flags/labels from the lesson. Keep the comment above each function describing when you would use that fact on-call. Run `./test.sh` until all 3 assertions pass.

Study loop

- **A (Drill):** make tests pass
- **B (Apply):** run 2–3 real commands from this lesson in your lab and note outputs
- **C (Challenge):** rewrite answers from memory next day

Do not open `solutions/20-tcpdump-basics/` until after an honest attempt.

Self-check

Answer without scrolling up:

1. Filter for DNS?
2. Flag to write pcap?
3. Read pcap without capture?

Further reading

- https://www.tcpdump.org/manpages/tcpdump.1.html